

DSA LAB-5 PRACTICE PROBLEMS

Linked Lists (**IMPORTANT: LL is NOT in the Lab Syllabus**)

Learning Resources

- [Striver Linked list playlist](#)
Note: Double linked list is not a part of the syllabus but it is recommended to practice it as well.
- [Linked List Data Structure | GFG](#)

Easy

1. [Middle of Linked List- Tortoise Hare Method](#)
2. [Reverse a linked list](#)
3. [Cycle Detection](#)

Medium

1. [Add 2 numbers](#)
2. [Design A Linked List](#) - Important for interviews
3. [Swapping Nodes in a Linked List](#)
4. [Double a Number Represented as a Linked List](#) - Unlike (1) number isn't stored in reverse order, so try to do it without reversing.

Hard

1. [Flattening of LL](#)
2. [LRU Cache Implementation](#) - Important Interview Question
3. [Reverse Nodes in k-Group](#)

Stacks

Learning Resources

- [Learn Stacks | Leetcode Discuss](#)
- [Stacks and Queues Playlist](#)
- [Learn about Standard Problems on Stacks](#) : It's really good btw

Easy

1. [Implement Stack using Queues](#)
2. [Valid Parentheses](#)
3. [Next Greater Element I](#) or [Nearest Smaller Values](#)

Medium

1. [Next Greater Element II](#)
2. [Infix to Postfix Conversion](#)
3. [The Celebrity Problem](#)
4. [Advertisement](#)
5. [Asteroid Collision](#)
6. [Mass of Molecule](#)

Hard

1. [Sum of Subarray Ranges](#)
2. [Trapping Rain Water](#)
3. [Maximal Rectangle](#)
4. [Remove K Digits](#)

Priority Queues

Learning Resources

- [Learn about Heaps and HeapSort](#)
- [Some Common Problems using Priority Queues](#)

Easy

1. [Does the array represent heap](#)
2. [Kth Smallest | Practice | GeeksforGeeks](#)
3. [Minimum Cost of ropes | Practice | GeeksforGeeks](#)
4. [Reorganize String - LeetCode](#)

Medium

1. [Stick Divisions | CSES](#)
2. [Room Allocation | CSES](#)
3. [Merge K Sorted Linked Lists](#) : Understand how to make a custom comparator for PQs (doesn't require linked list knowledge)
4. [Hand of Straights - LeetCode](#) : Just try to do it using PQs
5. [Task Scheduler](#) : The optimal solution doesn't need PQ necessarily, but try doing it using a PQ, so that u can build intuition.

Hard

1. [Find Median from Data Stream](#) : The question might feel easy, but this concept is really important
2. [Preparing For the Contest](#) : Ignore the rating
3. [Packing Under Range Regulations | Atcoder](#)

Recommended Basic Techniques for Upcoming Labs

You are advised to brush up on the following basic techniques, as most good problems will require familiarity with these. If any of the linked articles include topics that haven't been covered in labs (for example, queues), you can skip those sections. You only need the basic idea behind these techniques. If you've practiced enough problems so far, you've likely already encountered many of these in use.

- Two Pointers: <https://usaco.guide/silver/two-pointers>
- Prefix Sums: <https://usaco.guide/silver/prefix-sums>
- Prefix/Suffix Min/Max – computing min/max of $\text{array}[0..i]$ or $\text{array}[j..(n-1)]$ in $O(1)$
- Sliding Window: <https://usaco.guide/gold/sliding-window>

Coding and Debugging Guidelines for Labs and Practice

Please keep the following in mind while coding your solutions (in lab as well as while practicing):

1. Use print statements to trace values and understand what your code is actually doing.
In C++, use ``cerr`` for debugging since ``cerr`` output is not read by the autograder, so you can keep debug statements while testing.
In Python, use ``print(..., file=sys.stderr)``.
2. Write custom test cases, especially edge cases. Compile using ``g++`` and run your program manually by providing input yourself.
3. Isolate the issue by commenting out parts of the code and testing components independently:
 - Is your predicate function correct?
 - Are your ``lo`` and ``hi`` values in binary search valid?
 - Does each helper function behave as expected?
4. Learn how to compile by yourself from the terminal, and do not rely only on the autograder tool or VS Code. The compiler often provides clearer and more detailed error messages.

5. Be comfortable using the STL properly. Using `std::sort` (including with a custom comparator), reversing arrays, and similar operations using standard library APIs should be familiar to you by now.
6. Use meaningful variable names. It is difficult to understand what variables like `x`, `y`, `z` mean in your logic.
7. Format your code using `Ctrl + Shift + P` → `Format Document` in VS Code. Unformatted code is hard to debug.